

CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT  
CIÊNCIA DA COMPUTAÇÃO

**Relatório do trabalho final da disciplina de Estrutura de dados II, sobre operações de adição e remoção de em árvores AVL, rubro-negra e B.**

**Adriano Silva, Gustavo Gonçalves, Herton da Silveira, Matheus Freitas, Victor Alexandre Perim.**

**Joinville-SC**

**2024**

## Objetivo:

O objetivo deste trabalho consiste em analisar a complexidade algorítmica das operações de adição e remoção de nós (considerando o balanceamento) em árvores AVL, rubro-negra e B.

## Base:

Primeiramente foi feita a base do código, para padronizar e facilitar integração do algoritmo. Na base foram utilizadas todas as structs e as funções que o professor deu nos exemplos de árvores, as structs foram juntadas em uma só, e algumas das funções foram adaptadas pois são utilizadas em mais de uma árvore. Também foi feita uma função geradora de bases.

```
10 typedef enum
11 {
12     AVL,
13     RUBRO_NEGRA,
14     B
15 } tipoArvore;
16
17 typedef enum
18 {
19     VERMELHO,
20     PRETO
21 } Cor;
22
23 typedef struct No
24 {
25     struct No *pai;
26     struct No **filhos;
27     struct No *esq;
28     struct No *dir;
29     int *chaves;
30     int valor;
31     int total;
32
33     tipoArvore tipo;
34
35     union
36     {
37         struct
38         {
23 typedef struct No
24 {
25     union
26     {
27         struct
28         {
29             int altura;
30         } AVL;
31         struct
32         {
33             Cor cor;
34         } RUBRO_NEGRA;
35         struct
36         {
37             int ordem;
38         } B;
39     } caracteristica;
40 } No;
41
42 typedef struct Arvore
43 {
44     No *raiz;
45     No *nulo;
46     tipoArvore tipo;
47     int ordem;
48     int esforcoInserirAVL;
49     int esforcoRemoverAVL;
50     int esforcoInserirRN;
51     int esforcoRemoverRN;
52 } Arvore;
```

## Funções:

**incrementarEsforcoInsercao:** Tem como objetivo calcular e retornar o esforço de inserção em uma árvore. Dependendo do tipo da árvore (AVL ou Rubro-Negra), o esforço de inserção é calculado e retornado de forma diferente, para árvores AVL, o esforço é calculado usando o campo `esforcoInserirAVL`. Para árvores Rubro-Negra, o esforço é calculado usando o campo `esforcoInserirRB`. Caso a árvore não seja de nenhum desses tipos, a função retorna 0.

```
int incrementarEsforcoInsercao(Arvore *arvore, int valor)
{
    if (arvore->tipo == AVL)
    {
        arvore->esforcoInserirAVL = 0;
        adicionar(arvore, valor);
        return arvore->esforcoInserirAVL;
    }
    else if (arvore->tipo == RUBRO_NEGRA)
    {
        arvore->esforcoInserirRB = 0;
        adicionar(arvore, valor);
        return arvore->esforcoInserirRB;
    }
    return 0;
}
```

**incrementarEsforcoRemocao:** calcula o esforço necessário para remover um valor de uma árvore. Se a árvore for do tipo AVL, ela zera o contador de esforço de remoção, executa a remoção através da função remover e retorna o esforço registrado. Se a árvore for do tipo Rubro-Negra, o processo é similar, mas o esforço de remoção é registrado em uma variável diferente. Caso a árvore não seja de nenhum dos tipos, a função retorna 0. Essa função tem como objetivo calcular o custo da remoção com base no tipo da árvore.

```
int incrementarEsforcoRemocao(Arvore *arvore, int valor)
{
    if (arvore == NULL)
        return 0;

    if (arvore->tipo == AVL)
    {
        arvore->esforcoRemoverAVL = 0;
        remover(arvore, valor);
        return arvore->esforcoRemoverAVL;
    }
    else if (arvore->tipo == RUBRO_NEGRA)
    {
        arvore->esforcoRemoverRB = 0;
        remover(arvore, valor);
        return arvore->esforcoRemoverRB;
    }
    return 0;
}
```

**contabilizarEsforco:** Tem como objetivo contabilizar e retornar o esforço de inserção ou remoção em uma árvore. Se a árvore for do tipo AVL e o tipo de operação for inserção, ela incrementa e retorna o esforço de inserção (esforcoInserirAVL). Se for remoção, ela incrementa e retorna o esforço de remoção (esforcoRemoverAVL). Para árvores Rubro-Negra, o processo é semelhante, mas os esforços são contabilizados em esforcoInserirRB e esforcoRemoverRB. Se a árvore não for de nenhum dos tipos, a função retorna 0. A função realiza o incremento do esforço e retorna o valor atualizado de cada operação.

```
int contabilizarEsforco(Arvore *arvore, int tipo)
{
    if (arvore->tipo == AVL)
    {
        if (tipo == 0) // Esforço de Inserção
        {
            return arvore->esforcoInserirAVL++;
        }
        else if (tipo == 1) // Esforço de Remoção
        {
            return arvore->esforcoRemoverAVL++;
        }
    }
    else if (arvore->tipo == RUBRO_NEGRA)
    {
        if (tipo == 0) // Esforço de Inserção
        {
            return arvore->esforcoInserirRB++;
        }
        else if (tipo == 1) // Esforço de Remoção
        {
            return arvore->esforcoRemoverRB++;
        }
    }
    return 0;
}
```

## AVL

A Árvore AVL é uma árvore binária de busca BST que se mantém balanceada durante operações de inserção e remoção. Suas operações possuem complexidade  $O(\log n)$ .

A função de remoção remove um nó de uma árvore AVL enquanto mantém seu balanceamento. Primeiramente localiza o nó alvo de forma recursiva, e aplica duas estratégias: se o nó tem um ou nenhum filho, é substituído diretamente por esse filho ou por um nó nulo; se tem dois filhos, é substituído o valor pelo menor nó da subárvore direita, removendo recursivamente. Após a remoção, a altura dos nós ascendentes é atualizada.

```
No *adicionarNo(Arvore *arvore, No *no, int valor);
No *removerNo(Arvore *arvore, No *no, int valor);
void balancearAVL(Arvore *arvore, No *no, int tipoOperacao);
void percorrerInOrderAVL(No *no, void (*callback)(int));
No *rse(Arvore *arvore, No *no);
No *rsd(Arvore *arvore, No *no);
No *rde(Arvore *arvore, No *no);
No *rdd(Arvore *arvore, No *no);
int altura(No *no);
int fb(No *no);
void liberarNo(Arvore *arvore, No *no);
void liberarArvore(Arvore *arvore);
int contabilizarEsforco(Arvore *arvore, int tipo);
int incrementarEsforcoInsercao(Arvore *arvore, int valor);
int incrementarEsforcoRemocao(Arvore *arvore, int valor);
```

## Rubro Negra

Uma árvore rubro-negra é uma estrutura de dados balanceada binária que mantém seu equilíbrio utilizando regras específicas. Essas árvores garantem que a altura seja  $O(\log n)$ , o que é essencial para operações eficientes, como busca, inserção e remoção.

A remoção foi implementada com foco em três etapas principais: transplante, ajuste para manter as propriedades da rubro-negra e a liberação do nó removido.

As funções implementadas mantêm a árvore rubro-negra balanceada após inserções e remoções. **balancearRubroNegra** corrige desbalanceamentos realizando recolorações e rotações registrando os ajustes com **contabilizarEsforco**. **balancearRemocao** corrige desequilíbrios após remoções de nós pretos, analisando o irmão do nó substituto e aplicando **recolorações** e rotações, também contabiliza os esforços com **contabilizarEsforco**. **transplantar** substitui um nó por outro na árvore, ajustando os ponteiros. Já a função **removerNoRB** localiza e remove o nó alvo, substituindo por seu sucessor. Durante essas operações, esforços são contabilizados, garantindo que a árvore permaneça com altura  $O(\log n)$ .

```
void rotacionaresq(Arvore *arvore, No *no);
void rotacionardir(Arvore *arvore, No *no);
void balancearRubroNegra(Arvore *arvore, No *no);
No *localizar(Arvore *arvore, int valor);
void percorrerProfundidadeInOrder(Arvore *arvore, No *no, void (*callback)(int));
void liberarArvoreRN(Arvore *arvore, No *no);
void liberarArvoreCompleta(Arvore *arvore);
No *localizar(Arvore *arvore, int valor);
void visitar(int valor);
void removerRN(Arvore *arvore, No *z);
void removerValor(Arvore *arvore, int valor);
int incrementarEsforcoInsercaoRB(Arvore *arvore, int valor);
int incrementarEsforcoRemocaoRB(Arvore *arvore, int valor);
int contabilizarEsforcoRB(Arvore *arvore, int tipo);
Arvore *criarArvoreRB(tipoArvore tipo, int ordem);
No *adicionarNoRB(Arvore *arvore, No *no, int valor);
void balancearRemocao(Arvore *arvore, No *x);
```

## Árvore B

A árvore B é uma estrutura de dados balanceada usada para armazenar grandes volumes de dados de forma eficiente. Com múltiplos filhos por nó, ela minimiza a altura e reduz acessos ao disco. Cada nó armazena chaves ordenadas, facilitando buscas, inserções e remoções. Quando um nó excede sua capacidade, ele é dividido, mantendo o balanceamento. É amplamente usada em bancos de dados e sistemas de arquivos.

A função **removeChaveIterativa** localiza a chave a ser removida e, caso o nó seja folha, remove diretamente a chave e ajusta a árvore se necessário. Caso contrário a chave é substituída pelo sucessor e o processo continua. Após a remoção, a função **balancearAposRemocao** é chamada para garantir que os nós tenham o número mínimo de chaves, tentando emprestar uma chave de um irmão ou fundindo dois nós. A fusão é feita pela **fundirNos**, que move as chaves e filhos do nó irmão para o nó alvo. A **ehFolha** ajuda a verificar se um nó é folha ou não, determinando o tipo de operação a ser realizada. Dessa forma, as funções colaboram para manter a integridade da árvore B, garantindo que ela continue balanceada e eficiente para futuras operações.

```
void percorreArvore(No *no, void(visita)(int chave));
No *localizaChave(Arvore *arvore, int chave);
int pesquisaBinaria(No *no, int chave);
int encontrarIndiceChave(No *no, int chave);
int obterSucessor(No *no);
void balancearAposRemocao(Arvore *arvore, No *pai, int idx);
void fundirNos(Arvore *arvore, No *pai, int idx);
int removeChaveIterativa(Arvore *arvore, int chave);
void imprimirArvoreB(No *no, int nivel);
int ehFolha(No *no);
int transbordo(Arvore *arvore, No *no);
void adicionaChaveNo(No *no, No *direita, int chave);
No *divideNo(Arvore *arvore, No *no);
void adicionaChaveRecursivo(Arvore *arvore, No *no, No *novoFilho, int chave);
int adicionaChave(Arvore *arvore, int chave);
```



## Discussão Teórica

Os processos de inserção e remoção em árvores AVL, Rubro-Negra (RB) e B baseiam-se em buscas em profundidade seguidas de ajustes de balanceamento aplicados das folhas para a raiz. Como as rotações têm custo constante, o desempenho dessas operações é proporcional à altura da árvore.

A altura de uma árvore AVL com  $n$  nós no pior caso é aproximadamente  $1.44\log_2(n)$  o que reflete seu balanceamento rigoroso. Já as árvores RB têm uma altura maior, limitada a  $2\log_2(n)$ , o que as torna menos balanceadas que as AVL, mas com menor custo em operações de inserção e remoção devido à menor frequência de rotações.

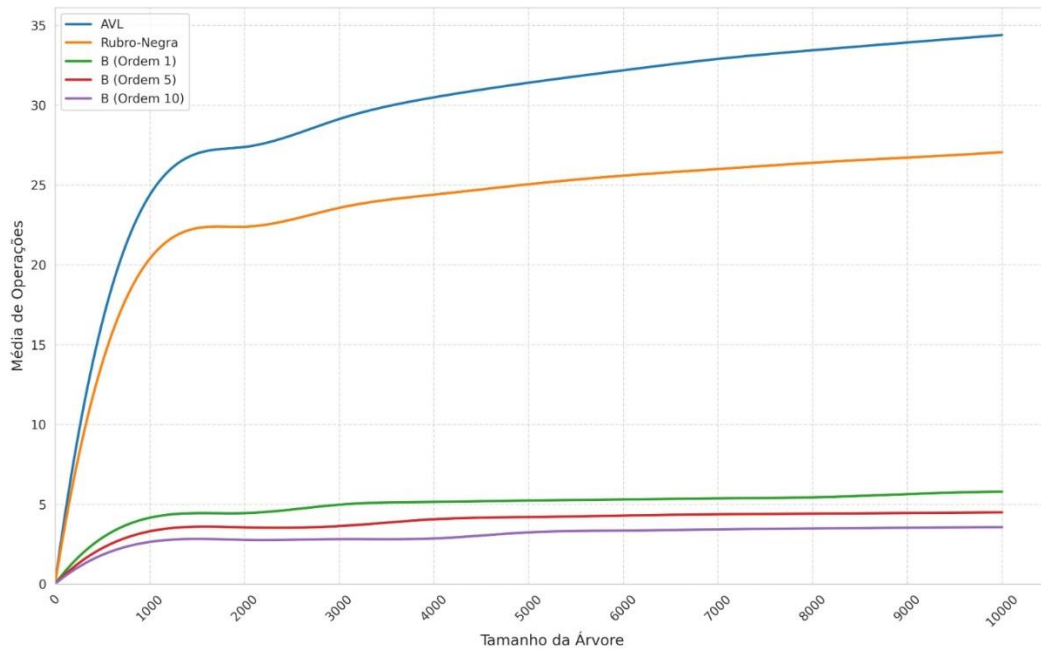
Por sua vez, árvores B, cuja altura é  $O(\log_M(n))$  dependendo do grau  $M$ , reduzem significativamente o número de níveis conforme  $M$  aumenta. No entanto, cada nó requer uma busca interna com custo  $O(\log_2(M))$  e balanceamentos que podem envolver movimentação de até  $M$  chaves. Ainda assim, com ordens altas, o número de transbordos diminui, reduzindo a necessidade de balanceamentos frequentes.

De forma geral, árvores AVL são ideais para cenários que exigem alta eficiência em buscas, enquanto árvores RB equilibram melhor custo de balanceamento e eficiência. Já as árvores B são mais indicadas para grandes volumes de dados, pois otimizam o acesso ao disco e minimizam a altura.

# Análise gráfica

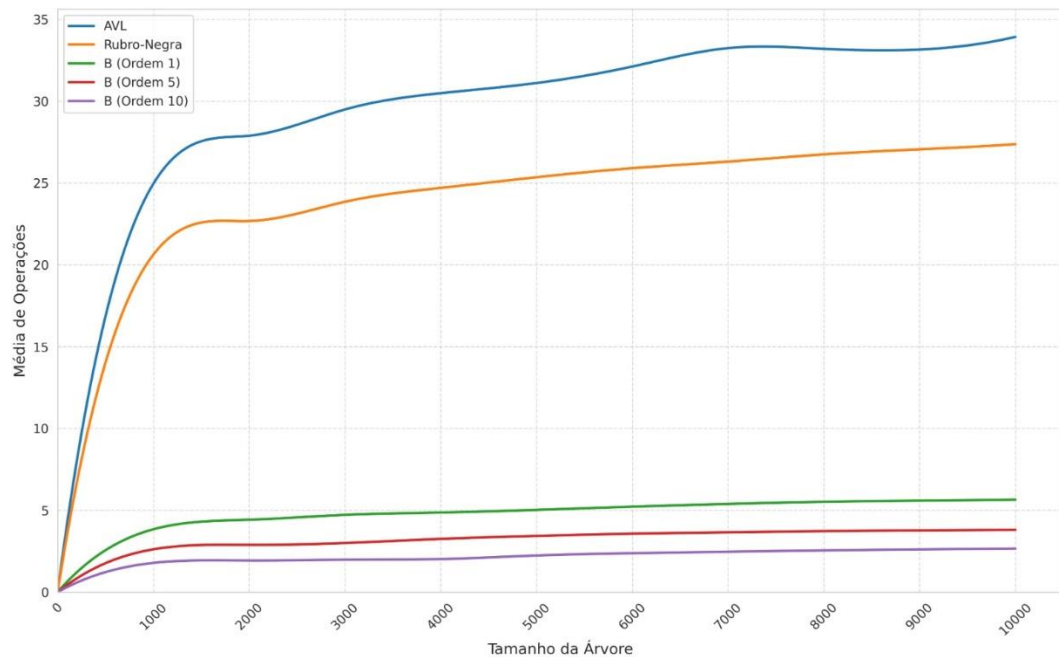
## Custo de inserção:

Comparação do Número Médio de Operações por Tipo de Árvore



## Custo de remoção:

Comparação do Número Médio de Operações por Tipo de Árvore



Os dados obtidos foram representados em nuvens de pontos que seguem o comportamento teórico esperado dos algoritmos analisados, todos apresentando complexidade logarítmica. Para melhorar a interpretação visual, os dados foram ajustados usando regressões logarítmicas do tipo  $\alpha \cdot \log_2(n)$ , que minimizam o desvio quadrático e fornecem curvas representativas de cada comportamento.

Observa-se que, em operações de inserção e remoção, as árvores AVL requerem mais acessos às chaves devido ao rigor de seu balanceamento. Árvores Rubro-Negra, por serem mais flexíveis, exibem menos acessos, ficando próximas ao desempenho das árvores B, especialmente em ordens maiores. Para árvores B, graus mais altos reduzem significativamente os acessos ao permitir que a altura da árvore diminua, tornando essas estruturas mais eficientes em operações de grande escala.

Assim, conclui-se que o desempenho em inserções e remoções é fortemente influenciado pelas características de balanceamento de cada estrutura: árvores AVL priorizam buscas rápidas com maior custo de manutenção, enquanto árvores RB e B oferecem operações mais eficientes no geral, especialmente em cenários com grande volume de dados.